

Choosing Between The Thinking And Non-Thinking Models

This guide explains when to use the `thinking` model versus the `non_thinking` model in `msa_zria`.

The short version:

- Use `non_thinking` when the task is narrow, direct, repetitive, and the right answer mostly comes from straightforward extraction or a short rule application.
- Use `thinking` when the task requires careful conflict resolution, multi-step reasoning, exception handling, or a defensible chain from evidence to decision.

What The Two Modes Mean

`non_thinking`

This is the narrow baseline path.

It is best when you want:

- fast structured parsing
- stable output contracts
- simple Pyro or ZRIA handoff
- low-cost inference for routine cases

It is not designed to spend tokens working through deeper ambiguity.

`thinking`

This is the specialist reasoning path.

It is best when you want:

- deeper reasoning over rules and evidence
- better handling of exceptions and overrides
- more robust code synthesis for hard cases
- stronger final evaluation on cases where a shallow answer is risky

It is trained from richer source cases that include:

- explicit rules
- verified claims
- exemplar reasoning traces
- verification checks

Practical Rule Of Thumb

Use `non_thinking` if a skilled operator could answer correctly in one short step after reading the case once.

Use **thinking** if a skilled operator would need to stop, compare rules, check exceptions, and justify the answer.

When To Choose **non_thinking**

Choose **non_thinking** when most of the work is one of these:

- extract a few fields from a message
- map an obvious issue to a known action
- apply a single dominant rule
- route routine cases through a stable workflow

Good Problem Characteristics

- The input is usually clean or predictable.
- The answer is usually determined by one or two visible facts.
- There are few meaningful policy conflicts or edge cases.
- A short explanation is enough.
- False positives from overthinking are a bigger risk than shallow reasoning.

Good Data Characteristics

- High volume of similar cases
- Consistent labels
- Short contexts
- Limited exception paths
- Minimal disagreement between annotators
- Targets mostly reflect direct extraction or standard response templates

Example Problems That Fit **non_thinking**

- Parse a support ticket into **device**, **issue**, **cause**, **severity**.
- Identify whether a refund request is inside or outside a fixed window when there are no exception clauses.
- Generate a standard Pyro sketch for a simple equipment failure pattern.
- Evaluate whether a candidate answer clearly matches an already-known resolution.

Example Training Data That Fits **non_thinking**

- standard support tickets with clean labels
- routine troubleshooting cases
- simple policy lookup examples
- canonical parse/code/evaluate records without long reasoning context

When To Choose **thinking**

Choose **thinking** when the model must reason before it can safely decide.

Good Problem Characteristics

- Multiple rules can apply, and they may conflict.
- The default rule is often overridden by a narrower exception.
- Important facts are distributed across message text, KG facts, and policy constraints.
- The final answer must be defensible, not just plausible.
- Missing one detail can change **resolved** into **escalate**.
- The code synthesis step must encode edge conditions, not just the happy path.

Good Data Characteristics

- Cases include exceptions, overrides, and conflict resolution.
- You can attach verified claims or evidence snippets.
- You can write down the rules that matter.
- You can describe what a correct reasoning trace must preserve.
- You can define verification checks for bad shallow answers.
- Hard cases are materially different from routine cases.

Example Problems That Fit **thinking**

- A refund case where the default opened-box policy says **manual review**, but a narrower hardware-failure exception may allow approval.
- A support escalation case where first-line remediation applies at first, but repeated failure means the case must escalate.
- A compliance case where one clause permits an action, another clause limits it, and the answer depends on precedence.
- A graph-backed decision where retrieved facts contain both normal-case evidence and explicit branch-specific overrides.

Example Training Data That Fits **thinking**

- source cases with explicit policy rules
- verified claims tied to evidence
- exemplar reasoning traces
- branch-sensitive KG cases
- escalation-boundary examples
- cases where shallow answers look reasonable but are wrong

More on Branch Aware/Sensitive KG Cases

In `msa_zria`, “branch-aware” comes from KG scope like **workspace**, **branch**, **commit**, and **as_of**. The same query can produce different answers if you run it against different graph branches, because each branch may contain different entities, relations, policies, or overrides.

A concrete example:

- On branch `main`, the graph says a device return with an opened box needs manual review.
- On branch `policy-review`, a new exception node says verified hardware failure within 30 days bypasses manual review.
- A `non_thinking` model may over-index on the first matching rule.

- A thinking model should notice both facts, resolve the conflict, and explain why the branch-specific exception wins.

Choosing By Failure Mode

Use **non_thinking** when the main failure risk is:

- unnecessary verbosity
- latency
- instability from over-generation
- solving routine work with too much reasoning

Use **thinking** when the main failure risk is:

- missing an exception
- ignoring a decisive claim
- applying the broad rule instead of the narrow override
- producing a superficially plausible but unsafe answer

Choosing By Training Objective

Train a **non_thinking** model when you want to optimize for:

- coverage of routine cases
- clean contract adherence
- high-throughput structured inference
- simple downstream orchestration

Train a **thinking** model when you want to optimize for:

- specialist reasoning depth
- exception-aware decisions
- better hard-case robustness
- stronger evidence-to-decision alignment

Concrete Split For **msa_zria**

Use **non_thinking** for:

- routine parse tasks
- direct classification-style evaluation
- standard support resolutions
- first-pass Pyro synthesis for simple patterns

Use **thinking** for:

- specialist support escalation
- policy override reasoning
- branch-aware graph reasoning with exceptions
- high-stakes evaluation where explanation quality matters

Final Guidance

Do not choose **thinking** just because a task is important.

Choose it when the task is structurally reasoning-heavy and your training data actually contains:

- rules
- exceptions
- evidence
- hard-case supervision

If the data is shallow, repetitive, and mostly direct-label mapping, training a **thinking** model will add cost without adding real capability.